COMPUTATIONAL COMPLEXITY: P, NP, NP-HARD & NP-COMPLETE

A Journey Through the Landscape of Hard Problems

Course: Theory of Computation / Algorithms • Duration: 2 Hours • Instructor: Murat Balci

> *"The question of whether P = NP is one of the seven Millennium Prize Problems, with a $1 million reward for its solution."*

ROADMAP FOR TODAY'S LECTURE

Part 1 (30 min)

Complexity Foundations

Understanding time complexity and the class P.

Part 2 (30 min)

The Class NP

The art of solving versus verifying.

Part 3 (30 min)

NP-Complete & NP-Hard

The hardest problems in NP and beyond.

Part 4 (30 min)

The P vs NP Question

The million-dollar mystery and its real-world implications.

By the end, students will be able to classify problems into complexity classes, understand polynomial-time reductions, and appreciate why P vs NP matters.

PART 0 · MOTIVATION

Complexity Theory Shapes the Real World

Understanding what computers *can* and *cannot* do efficiently has profound implications across technology, science, and society.

CRYPTOGRAPHY

Every time you shop online or send a message, your security depends on certain problems being **hard to solve**. If P = NP, all modern encryption breaks.

RSA relies on factoring being hard

OPTIMIZATION

Logistics companies route millions of packages daily. Knowing a problem is NP-Hard tells engineers to use **approximations** instead of searching for perfect solutions.

UPS saves $400M/yr with heuristics

ARTIFICIAL INTELLIGENCE

Many AI planning and learning tasks are NP-Hard. Understanding complexity helps us design **tractable subproblems** and know when to accept approximate answers.

Constraint satisfaction in AI planning

PART 1 · FOUNDATIONS

How We Measure Algorithm Efficiency

We describe how an algorithm's running time grows as the input size **n** increases, using **Big-O notation**.

KEY IDEA

$f(n) = O(g(n))$

Ignore constants and lower-order terms. Focus on the **dominant growth rate** as $n \to \infty$.

Big-O gives us a language to compare algorithms *independent of hardware*. An $O(n^2)$ algorithm will always eventually lose to an $O(n \log n)$ algorithm for large enough n.

> *"The critical boundary in complexity theory is between **polynomial** time (tractable) and **exponential** time (intractable)."*

NAME	BIG-O	N = 10	N = 100	N = 1,000
Constant	$O(1)$	1	1	1
Logarithmic	$O(\log n)$	3	7	10
Linear	$O(n)$	10	100	1,000
Log-linear	$O(n \log n)$	33	664	9,966
Quadratic	$O(n^2)$	100	10,000	1,000,000
Cubic	$O(n^3)$	1,000	10^6	10^9
Exponential	$O(2^n)$	1,024	10^{30}	10^{301}
Factorial	$O(n!)$	3.6M	10^{158}	10^{2567}

PART 1 · FOUNDATIONS

Why the Polynomial Boundary Matters So Much

A small change in growth rate creates an astronomical difference at scale.

GROWTH RATE	N = 10	N = 20	N = 50	N = 100
$O(n)$	10	20	50	100
$O(n^2)$	100	400	2,500	10,000
$O(n^3)$	1,000	8,000	125,000	1,000,000
$O(2^n)$	1,024	1,048,576	$\sim 10^{15}$	$\sim 10^{30}$
$O(n!)$	3,628,800	$\sim 10^{18}$	$\sim 10^{64}$	$\sim 10^{158}$

At $n = 50$, an $O(n^3)$ algorithm finishes in milliseconds. An $O(2^n)$ algorithm would take over 11 days at 1 billion ops/sec. At n = 100, it would take longer than the age of the universe.

THE WALL

Where Exponential Hits

Even the world's fastest supercomputer cannot overcome exponential growth. Doubling speed only lets you solve a problem *one size larger*.

10^{30}

OPERATIONS FOR 2^{100}

That is 1,000,000,000,000,000,000,000,000,000,000 — more than the number of atoms in a human body.

This is why we draw the line at **polynomial time**. It separates the *feasible* from the *impossible*.

PART 1 · FOUNDATIONS

Complexity Classes Are Defined for Decision Problems

DEFINITION

A **decision problem** is a computational problem whose output is always a single bit: **YES** or **NO**.

Every input maps to exactly one of two answers — there is no ambiguity, no partial credit, no "maybe."

WHY DECISION PROBLEMS?

Using YES/NO answers makes it possible to formally compare problems. It strips away output-formatting details and focuses purely on the *inherent difficulty* of computation.

OPTIMIZATION → DECISION CONVERSION

OPTIMIZATION	→	DECISION
"What is the **shortest** path from A to B?" *Output: a number (e.g., 42)*	→	"Is there a path from A to B with length ≤ **k**?" *Output: YES or NO*
"What is the **minimum** tour visiting all cities?" *Output: a route + distance*	→	"Is there a tour of all cities with distance ≤ **k**?" *Output: YES or NO*
"What is the **maximum** value in a knapsack?" *Output: a number (e.g., $85)*	→	"Can we pack items worth ≥ **v** within weight ≤ **W**?" *Output: YES or NO*

Key Insight

The decision version is *never harder* than the optimization version. If you can solve the optimization, you can answer the decision question trivially.

PART 1 · CLASS P

P = The Set of Decision Problems Solvable in Polynomial Time

FORMAL DEFINITION

P = { L | there exists a **deterministic** Turing machine M and a polynomial p(n) such that M decides L in time O(p(n)) for all inputs of size n }

IN PLAIN ENGLISH

A problem is in **P** if there exists an algorithm that can **always find the correct answer** in a number of steps bounded by a polynomial function of the input size — such as n^2, n^3, or n^{10}.

KEY PROPERTIES OF P

01 Deterministic

No guessing needed. The algorithm follows a single, definite path of computation.

02 Scalable

Doubling the input size increases runtime by a bounded, predictable factor — not an explosion.

03 Closed Under Composition

Running one P algorithm after another still yields a polynomial-time algorithm.

04 Subset of NP

Every problem in P is also in NP. If you can solve it fast, you can verify it fast.

▸ P is often called the class of "tractable" or "efficiently solvable" problems.

PART 1 · CLASS P

Familiar Problems That Live in Class P

PROBLEM	WHAT IT ASKS	ALGORITHM	TIME COMPLEXITY
Sorting	Arrange n elements in order	Merge Sort	$O(n \log n)$
Shortest Path	Find the cheapest route between two nodes in a weighted graph	Dijkstra's Algorithm	$O(E + V \log V)$
2-Coloring	Color a graph with 2 colors so no adjacent vertices share a color	BFS / DFS	$O(V + E)$
Euler Path	Find a path visiting every edge exactly once	Degree Check + Hierholzer	$O(V + E)$
Primality Test	Is a given number n prime?	AKS Algorithm	$O(\log^6 n)$
2-SAT	Satisfy a Boolean formula where each clause has exactly 2 literals	Implication Graph + SCC	$O(V + E)$

Key insight: All these problems have known polynomial-time algorithms. They are "tractable" — we can solve them efficiently even for very large inputs.

PART 1 · WORKED EXAMPLE

Walking Through Dijkstra's Algorithm

GRAPH — 5 NODES, 6 EDGES

A → B : 4
A → C : 2
C → B : 1
C → D : 3
B → E : 3
D → E : 1

DECISION PROBLEM

Is there a path from A to E with total distance ≤ 6?

SHORTEST PATH

A → C → B → E

TOTAL DISTANCE

6 ≤ 6 YES

ALGORITHM EXECUTION — STEP BY STEP

STEP	VISIT	DIST[A]	DIST[B]	DIST[C]	DIST[D]	DIST[E]	ACTION
Init	—	0	∞	∞	∞	∞	Set source = 0
1	A	0 ✓	4	2	∞	∞	B=0+4, C=0+2
2	C	0 ✓	3	2 ✓	5	∞	B=min(4,3), D=2+3
3	B	0 ✓	3 ✓	2 ✓	5	6	E=3+3=6
4	D	0 ✓	3 ✓	2 ✓	5 ✓	6	E=min(6,6) no change
5	E	0 ✓	3 ✓	2 ✓	5 ✓	6 ✓	All nodes visited

NODES VISITED

5 nodes, 6 edge checks

TIME COMPLEXITY

O(V + E) — Polynomial ∈ P

PART 2 · CLASS NP

NP = Decision Problems Whose YES Answers Can Be Verified in Polynomial Time

FORMAL DEFINITION

NP = { L | there exists a verifier V and a polynomial p(n) such that for every YES instance x, there exists a **certificate** c with |c| ≤ p(|x|) and V(x, c) accepts in time O(p(|x|)) }

THE CERTIFICATE (WITNESS)

A **certificate** is a piece of evidence that lets you **quickly check** a YES answer — without re-solving the entire problem from scratch.

- Sudoku → the filled grid
- SAT → the variable assignment
- Hamiltonian Cycle → the cycle itself
- Subset Sum → the subset

✓ NP MEANS

- → **Nondeterministic Polynomial** time
- → Solutions can be **verified** in polynomial time
- → **Includes all of P** — every P problem is also in NP

✗ NP DOES NOT MEAN

- → "Not Polynomial" — this is the #**1 misconception**
- → That the problem **cannot** be solved in polynomial time
- → That the problem requires **exponential** time

? The central question of NP: just because you can *check* an answer quickly, does that mean you can *find* it quickly? This is the essence of the P vs NP problem.

PART 2 · CLASS NP

Finding a Needle in a Haystack vs. Confirming It Is a Needle

SOLVING (FINDING)	VS	VERIFYING (CHECKING)
🧩 JIGSAW PUZZLE Assemble a **1,000-piece puzzle** from a jumbled pile of pieces with no picture on the box. Hours to days	vs	✅ JIGSAW PUZZLE Look at a **completed puzzle** and confirm every piece fits with its neighbors. A quick glance
🔒 COMBINATION LOCK Crack a **4-digit lock** by trying all 10,000 possible combinations one by one. Up to 10,000 tries	vs	🔑 COMBINATION LOCK Someone hands you the code **"7-2-9-4"**. Just enter it and see if the lock opens. 1 single try
📝 CROSSWORD PUZZLE Fill in a **blank crossword** from scratch, finding words that satisfy every intersection. Requires deep search	vs	📋 CROSSWORD PUZZLE Given a **filled-in crossword**, check each word against the clue list. Read and compare

❓ Is creation *fundamentally harder* than verification? If solving is always harder than checking, then **P ≠ NP**. If not — if every efficiently checkable problem is also efficiently solvable — then **P = NP**.

PART 2 · CLASS NP

Sudoku: Easy to Check, Hard to Solve

! SOLVING THE PUZZLE

POTENTIALLY EXPONENTIAL

5	3	?	?	7	?	?	?	?
6	?	?	1	9	5	?	?	?
?	9	8	?	?	?	?	6	?
8	?	?	?	6	?	?	?	3
4	?	?	8	?	3	?	?	1
7	?	?	?	2	?	?	?	6
?	6	?	?	?	?	2	8	?
?	?	?	4	1	9	?	?	5
?	?	?	?	8	?	?	7	9

51
EMPTY CELLS

~10³⁸
POSSIBLE FILLINGS

Backtrack
STRATEGY

✓ VERIFYING A SOLUTION

POLYNOMIAL – O(N²)

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

- ✓ **9 rows** — each contains digits 1–9 exactly once
- ✓ **9 columns** — each contains digits 1–9 exactly once
- ✓ **9 boxes** — each 3×3 block contains 1–9 exactly once

NP

Generalized n×n Sudoku is **NP-Complete**. Finding a solution may require exponential backtracking, but given a completed grid (the **certificate**), verification takes only O(n²) — just scan each row, column, and box. This asymmetry is the essence of NP.

PART 2 · CLASS NP

Problems Where Checking Is Easy but Finding Is Hard

Each of these problems is in NP — solutions can be verified in polynomial time, even though no fast solving algorithm is known.

PROBLEM 01

Graph 3-Coloring

FIND — Try all possible colorings of V vertices with 3 colors

$O(3^V)$ — exponential

CHECK — For each edge, verify endpoints differ in color

$O(E)$ — polynomial

PROBLEM 02

Subset Sum

FIND — Try all 2^n subsets of the set to match the target

$O(2^n)$ — exponential

CHECK — Add the proposed subset's elements, compare to target

$O(n)$ — polynomial

PROBLEM 03

Boolean SAT

FIND — Try all 2^n truth assignments for n variables

$O(2^n)$ — exponential

CHECK — Plug in the assignment, evaluate each clause

$O(m)$ — polynomial

PROBLEM 04

Hamiltonian Cycle

FIND — Try all permutations of vertices for a valid cycle

$O(n!)$ — factorial

CHECK — Walk the proposed cycle, verify each edge exists

$O(V)$ — polynomial

PROBLEM 05

Clique

FIND — Search all subsets of k vertices for a complete subgraph

$O(C(n,k))$ — exponential

CHECK — Verify every pair in the proposed set is connected

$O(k^2)$ — polynomial

PROBLEM 06

Vertex Cover

FIND — Search all subsets of k vertices that touch every edge

$O(C(n,k))$ — exponential

CHECK — For each edge, verify at least one endpoint is in the set

$O(E)$ — polynomial

PATTERN

In every case, **finding** the solution requires exploring an exponential search space, but **checking** a proposed solution takes only polynomial time. This asymmetry defines NP.

PART 2 · WORKED EXAMPLE

Subset Sum: The Certificate Verification Process

INPUT — SET S AND TARGET T

3 | 7 | 1 | 8 | 4 | 12 | 5

Target T = **20**

CERTIFICATE — PROPOSED SUBSET

3 | 8 | 4 | 5

Claim: these elements sum to **20**

VERIFICATION PIPELINE — 3 STEPS, ALL POLYNOMIAL

SUBSET CHECK 01

Verify every element in the certificate actually belongs to the original set S.

3 ∈ S? ✓
8 ∈ S? ✓
4 ∈ S? ✓
5 ∈ S? ✓

PASS — O(k·n) time

SUM COMPUTATION 02

Add all elements in the proposed subset together to compute the total.

  3
+ 8 = 11
+ 4 = 15
+ 5 = **20**

PASS — O(k) time

TARGET COMPARISON 03

Compare the computed sum against the target value T.

Sum = **20**
Target = **20**
20 = 20 ? **YES** ✓

PASS — O(1) time

VERIFICATION VERDICT

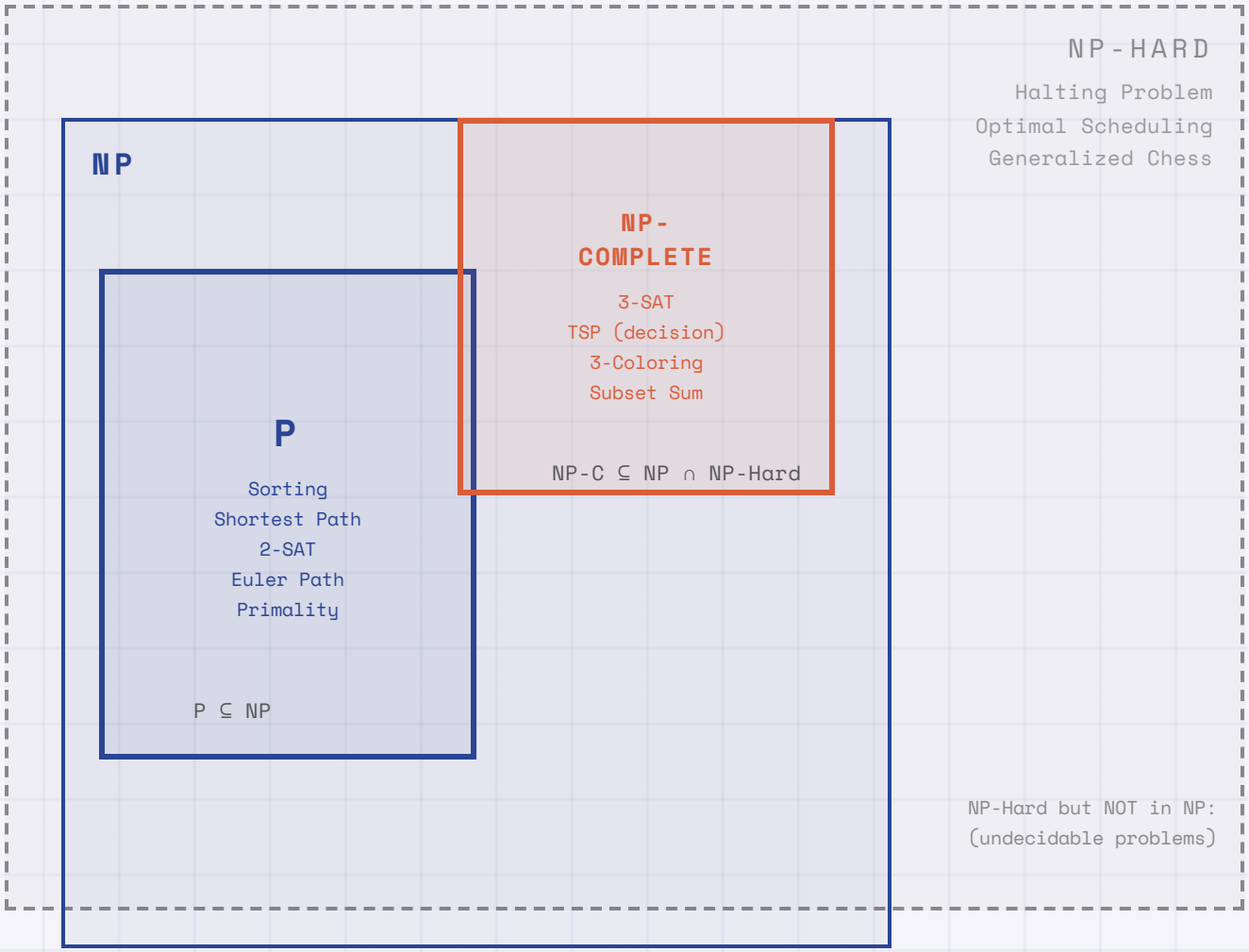
All 3 checks passed. The certificate is valid. Total verification time: O(n) — polynomial. This confirms Subset Sum is in NP.

CONTRAST — FINDING THE CERTIFICATE

Without the certificate, a brute-force solver would need to examine up to $2^7 = 128$ subsets. For n = 50 items, that becomes $2^{50} \approx 10^{15}$ subsets.

PART 2 · THE BIG PICTURE

Visualizing the Relationship Between P and NP



REGION KEY

P
Solvable *and* verifiable in polynomial time. The "easy" problems.

NP
Verifiable in polynomial time. May or may not be solvable efficiently.

NP-Complete
The hardest problems *within* NP. Both in NP and NP-Hard.

NP-Hard
At least as hard as NP-Complete. May not even be in NP (may be undecidable).

CONTAINMENT RULES

P ⊆ NP

NP-Complete = NP ∩ NP-Hard

NP-Complete ⊆ NP-Hard

OPEN QUESTION

Does **P = NP**? If yes, the P and NP-Complete zones merge — every verifiable problem becomes efficiently solvable.

PART 3 · REDUCTIONS

Reductions: Comparing Problem Difficulty

FORMAL DEFINITION

$$A \leq_P B$$

Problem **A** reduces to problem **B** in polynomial time if there exists a function **f** computable in polynomial time such that $x \in A$ if and only if $f(x) \in B$.

THE CORE INTUITION

A reduction says: *"If I could solve B, I could use that to solve A."* It means **B is at least as hard as A** — solving B would give us a way to solve A too.

Think of it as a **translator**: convert any A-question into a B-question, solve it, and translate the answer back.

THE REDUCTION PIPELINE

STEP 1
Instance of A
The problem you want to solve

→

STEP 2
Transform f(x)
Polynomial-time conversion

→

STEP 3
Instance of B
Equivalent question in B's language

→

STEP 4
Solve B
Use B's algorithm

→

RESULT
Answer for A
YES / NO

KEY INSIGHT 01 — FORWARD

If B is easy, then A is easy

If we have a fast algorithm for B, we can solve A by reducing it to B first. The total time is: **poly(transform)** + **poly(solve B)** = **polynomial**.

KEY INSIGHT 02 — CONTRAPOSITIVE

If A is hard, then B is hard

If A has no fast algorithm, then B cannot have one either — because a fast B would give us a fast A. **Hardness transfers upward.**

PART 3 · WORKED EXAMPLE

Reducing Independent Set to Vertex Cover

INDEPENDENT SET

Find a set of **k vertices** where **no two are adjacent** (no edge connects any pair).

Given (G, k): Is there S ⊆ V,
|S| ≥ k, s.t. ∀ u,v ∈ S: **(u,v) ∉ E** ?

⟺

VERTEX COVER

Find a set of **k vertices** that **touches every edge** (every edge has at least one endpoint in the set).

Given (G, k): Is there C ⊆ V,
|C| ≤ k, s.t. ∀ (u,v) ∈ E: **u ∈ C or v ∈ C** ?

REDUCTION S is an **Independent Set** of size k ⟺ V \ S is a **Vertex Cover** of size n - k | Transform: (G, k) → (G, **n - k**)

GRAPH G — 6 NODES, 7 EDGES

(A,B) (A,C) (B,C)
(B,D) (C,E) (D,E)
(E,F)

INDEPENDENT SET (K = 3)

S = { A, D, F }

Edge (A,D)? Not in E ✓
Edge (A,F)? Not in E ✓
Edge (D,F)? Not in E ✓

No pair is adjacent — valid!

VERTEX COVER (N-K = 3)

C = V \ S = { B, C, E }

(A,B) ✓ (A,C) ✓ (B,C) ✓
(B,D) ✓ (C,E) ✓ (D,E) ✓
(E,F) ✓

Every edge is covered — valid!

THE REDUCTION IN ACTION

IndependentSet(G, 3)
→ transform: keep G, set k' = 6 - 3 = 3
→ **VertexCover**(G, 3)
→ Answer: **YES** → return **YES**

WHY THIS WORKS

If no edge connects any pair in S, then **every edge must have an endpoint outside S** — which is exactly V \ S. The complement of an independent set is always a vertex cover, and vice versa. The transformation takes **O(1)** time — just change k to n − k.

PART 3 · NP-HARD

NP-Hard Problems: The Hardest Problems (and Beyond)

FORMAL DEFINITION

A problem L is **NP-Hard** if:
for **every** problem L' ∈ NP,
L' $\leq_p$ L

IN PLAIN ENGLISH

A problem is **NP-Hard** if it is **at least as hard** as every problem in NP. Every NP problem can be transformed into it in polynomial time — it is a "universal" problem that captures all of NP's difficulty.

! CRITICAL DISTINCTION

NP-Hard does **NOT** require the problem to be in NP. Some NP-Hard problems are so hard they are **undecidable** — no algorithm of any speed can solve them. NP-Hard is a **lower bound** on difficulty, not an upper bound.

NP-HARD AND IN NP → NP-COMPLETE

These are the "sweet spot" — hard but still verifiable

3-SAT
Satisfy a formula with 3-literal clauses
NP-Complete

TSP (Decision)
Tour all cities within distance k?
NP-Complete

Graph 3-Coloring
Color vertices with 3 colors, no conflicts
NP-Complete

Subset Sum
Find subset summing to target
NP-Complete

NP-HARD BUT NOT IN NP

These are even harder — often undecidable or beyond NP

Halting Problem
Does a program ever stop?
Undecidable

Optimal TSP
Find the actual shortest tour
Optimization

Generalized Chess
Perfect play on n×n board
EXPTIME-Hard

QBF (∀∃-SAT)
Quantified Boolean formulas
PSPACE-Complete

PART 4 · NP-COMPLETE

NP-Complete = NP-Hard ∩ NP

The Hardest Problems *Inside* NP

DEFINITION

NP-Complete

=

CONDITION 2

NP-Hard

∩

CONDITION 1

NP

01 MUST BE IN NP

The problem must have **polynomial-time verification**. Given a proposed solution (certificate), a verifier can check its correctness in polynomial time.

- → Solutions are short (polynomial size)
- → Checking is fast (polynomial time)

02 MUST BE NP-HARD

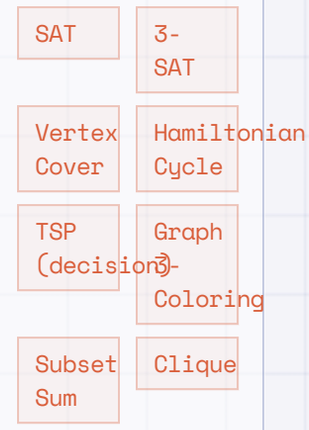
Every problem in NP can be reduced to it in polynomial time. It is at least as hard as every other problem in NP — a "universal" problem.

- → $\forall L \in NP: L \leq_p$ this problem
- → Hardness proven via reductions

! THE DOMINO EFFECT — WHY THIS MATTERS

If **any single** NP-Complete problem is solved in polynomial time, then **every problem in NP** can be solved in polynomial time — because they all reduce to it. This would prove **P = NP**. Conversely, proving one NP-Complete problem has no polynomial algorithm proves **P ≠ NP**.

CANONICAL NP-COMPLETE PROBLEMS



SUMMARY · QUICK REFERENCE

P vs NP vs NP-Hard vs NP-Complete at a Glance

CLASS	DEFINITION	SOLVABLE IN POLY TIME?	VERIFIABLE IN POLY TIME?	KEY EXAMPLES
P Polynomial Time	Problems solvable by a **deterministic** algorithm in polynomial time $O(n^k)$.	**YES** by definition	**YES** P ⊆ NP	**Sorting**, Shortest Path, 2-SAT, Primality, Euler Path, MST
NP Nondeterministic Polynomial	Problems whose solutions can be **verified** in polynomial time given a certificate.	**?** open question	**YES** by definition	**All of P**, plus Subset Sum, TSP (decision), Hamiltonian Cycle
NP-Hard NP-Hardness	Problems **at least as hard** as every NP problem. Every NP problem reduces to it.	**?** likely no	**NOT REQ.** may be undecidable	**Halting Problem**, Optimal TSP, Gen. Chess, all NP-Complete
NP-Complete NP ∩ NP-Hard	In NP **and** NP-Hard. The hardest problems that are still verifiable.	**?** P = NP?	**YES** in NP	**3-SAT**, Vertex Cover, 3-Coloring, Knapsack (decision)

CONTAINMENT RELATIONSHIPS

- **P** ⊆ **NP**
- **NP-C** ⊆ **NP**
- **NP-C** ⊆ **NP-Hard**
- **NP-C** = **NP** ∩ **NP-Hard**
- If **P** = **NP**, then **NP-C** ⊆ **P**
- **NP-Hard** ⊄ **NP** (in general)

QUICK MEMORY AID

P = easy to **solve**. **NP** = easy to **check**. **NP-Hard** = at least as hard as NP (maybe harder). **NP-Complete** = the hardest problems you can still **check** quickly — the boundary between tractable and intractable.

PART 4 · THE FIRST NP-COMPLETE PROBLEM

SAT Was the First Problem Proven NP-Complete (1971)

THEOREM

The Cook-Levin Theorem

The Boolean Satisfiability Problem (SAT) is NP-Complete. SAT is in NP, and every problem in NP reduces to SAT in polynomial time.

Stephen Cook (1971) · Leonid Levin (1973)

WHAT IS SAT?

Given a Boolean formula in **Conjunctive Normal Form** (CNF), is there an assignment of TRUE/FALSE to variables that makes the **formula TRUE**?

$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3)$ Try: x_1=T, x_2=T, x_3=T — TRUE

Answer: **YES**

WHY THIS CHANGED EVERYTHING

01 **First concrete NP-Complete problem.** SAT gave a real example that anchored the theory.

02 **Universal encoding power.** Any nondeterministic computation can be encoded as a SAT formula.

03 **Unlocked the reduction chain.** Other problems could be shown NP-Complete by reducing from SAT.

04 **Practical impact.** Modern SAT solvers power verification, planning, and testing.

CORE PROOF IDEA

Model each step of a polynomial-time verifier with Boolean constraints; assembling these constraints into a CNF yields a formula satisfiable iff the verifier accepts.

1971
Cook-Levin Theorem
SAT proven NP-Complete — the first domino.

→

1972
Karp's 21 Problems
Karp reduces SAT to 21 classic problems.

→

Today
Thousands Known
Over 3,000 problems proven NP-Complete via reductions.

PART 4 · WORKED EXAMPLE

Solving and Verifying a SAT Instance

INPUT — CNF FORMULA Φ (4 CLAUSES, 4 VARIABLES)

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_4) \wedge (x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee x_4)$$

4 clauses, each with **3 literals** — this is a **3-SAT** instance

CERTIFICATE — VARIABLE ASSIGNMENT

x_1 = TRUE	x_2 = TRUE
x_3 = FALSE	x_4 = TRUE

CLAUSE-BY-CLAUSE VERIFICATION

CLAUSE 1 — TRUE ✓

$(x_1 \vee \neg x_2 \vee x_3)$

↓ substitute

$(T \vee \neg T \vee F)$

↓ evaluate

$(T \vee F \vee F) = \text{TRUE}$

CLAUSE 2 — TRUE ✓

$(\neg x_1 \vee x_2 \vee \neg x_4)$

↓ substitute

$(\neg T \vee T \vee \neg T)$

↓ evaluate

$(F \vee T \vee F) = \text{TRUE}$

CLAUSE 3 — TRUE ✓

$(x_2 \vee x_3 \vee x_4)$

↓ substitute

$(T \vee F \vee T)$

↓ evaluate

$(T \vee F \vee T) = \text{TRUE}$

CLAUSE 4 — TRUE ✓

$(\neg x_1 \vee \neg x_3 \vee x_4)$

↓ substitute

$(\neg T \vee \neg F \vee T)$

↓ evaluate

$(F \vee T \vee T) = \text{TRUE}$

VERIFICATION VERDICT

$$\varphi = \text{TRUE} \wedge \text{TRUE} \wedge \text{TRUE} \wedge \text{TRUE} = \text{TRUE}$$

All 4 clauses evaluate to TRUE. The certificate is **valid** — the formula is **satisfiable**.

Verification time: $O(m \times k) = O(4 \times 3) = 12$ operations — polynomial ✓

FINDING VS. CHECKING

Checking the certificate took **12 operations**. But **finding** it without a hint means trying variable assignments until one works.

4 variables → 2^4 = **16** possible assignments

20 variables → 2^{20} = **1,048,576**

100 variables → $2^{100} \approx 10^{30}$ assignments

PART 4 · NP-COMPLETE GALLERY

A Tour of the Most Important NP-Complete Problems

Every problem below is **equivalent in difficulty** — solving any one of them in polynomial time solves them all.

01 3-SAT

Satisfy a Boolean formula where every clause has exactly 3 literals.

Input: CNF formula, 3 literals/clause
Ask: Assignment making all clauses TRUE?

LOGIC Karp, 1972

02 Traveling Salesman

Visit every city exactly once and return home within a distance budget.

Input: n cities, distances, budget k
Ask: Tour of length ≤ k exists?

GRAPH Karp, 1972

03 Graph 3-Coloring

Color every vertex with one of 3 colors so no adjacent vertices share a color.

Input: Undirected graph G
Ask: Valid 3-coloring exists?

GRAPH Karp, 1972

04 Vertex Cover

Find a set of k vertices that touches every edge in the graph.

Input: Graph G, integer k
Ask: Cover of size ≤ k exists?

GRAPH Karp, 1972

05 Hamiltonian Cycle

Find a cycle that visits every vertex in the graph exactly once.

Input: Undirected graph G
Ask: Hamiltonian cycle exists?

GRAPH Karp, 1972

06 Subset Sum

Find a subset of integers that adds up to a given target value T.

Input: Set of integers S, target T
Ask: Subset summing to T exists?

NUMBER Karp, 1972

07 Clique

Find a complete subgraph of k vertices where every pair is connected.

Input: Graph G, integer k
Ask: Clique of size ≥ k exists?

GRAPH Karp, 1972

08 Knapsack (Decision)

Pack items into a weight-limited bag to reach a minimum total value.

Input: Items (weight, value), W, V
Ask: Total value ≥ V within weight W?

NUMBER Karp, 1972

09 Set Cover

Select at most k subsets whose union covers the entire universe.

Input: Universe U, subsets $S_1 \ldots S_m$, k
Ask: ≤ k subsets covering U exist?

SET Karp, 1972

PART 4 · REAL-WORLD NP-COMPLETE

Packing Your Suitcase Is NP-Complete

You're flying for a weekend trip. Your suitcase holds **15 kg max**. Each item has a weight and a "joy score" (how much you'd enjoy having it). Goal: reach at least **30 joy points** without exceeding the weight limit.

ITEM	WEIGHT	JOY	PACK?
💻 **Laptop**	3 kg	9	✓
📚 Books	5 kg	7	–
📷 **Camera**	2 kg	8	✓
👕 **Clothes**	4 kg	6	✓
🍪 **Snacks**	1 kg	3	✓
🎲 **Board Game**	3 kg	5	✓
💨 Hair Dryer	2 kg	2	–
👟 Shoes	4 kg	4	–

CERTIFICATE VERIFICATION

PROPOSED PACKING (CERTIFICATE)

Laptop | Camera | Clothes | Snacks | Board Game

STEP 1 — WEIGHT CHECK

3 + 2 + 4 + 1 + 3 = **13 kg**

13 ≤ 15? **YES**

PASS — within limit

STEP 2 — JOY CHECK

9 + 8 + 6 + 3 + 5 = **31 joy**

31 ≥ 30? **YES**

PASS — target met

BUT IS THIS THE BEST PACKING?

We verified this packing **meets the target** — that's the **decision** version (NP). But finding the **absolute best** packing (optimization) is even harder — that's **NP-Hard**, and the answer isn't even easy to verify as "optimal."

WHY KNAPSACK IS IN NP

Given a proposed packing, we just **add up weights** and **add up values** — two simple sums. Verification takes **O(n)** time. Easy to check, even for thousands of items.

WHY KNAPSACK IS NP-HARD

With 8 items, there are $2^8 = 256$ possible subsets. With 50 items: $2^{50} \approx 10^{15}$. No known algorithm avoids checking an exponential number of combinations in the worst case.

PART 5 · THE THIN LINE

Small Modifications Can Push Problems from P to NP-Complete

The boundary between tractable and intractable is **razor-thin**. Changing a single number or word can flip complexity.

IN P — EFFICIENTLY SOLVABLE	THE SMALL CHANGE	NP-COMPLETE — INTRACTABLE
2-SAT Boolean formula with ≤2 literals per clause; solved via implication graphs. P — O(n)	→ Change "2" to "3" literals	**3-SAT** Boolean formula with 3 literals per clause; canonical NP-Complete. NP-Complete
2-Coloring Graph coloring with 2 colors (bipartiteness check). P — O(V+E)	→ Change "2" to "3" colors	**3-Coloring** Graph coloring with 3 colors; becomes intractable. NP-Complete
Euler Path Traverse every edge exactly once (degree checks). P — O(E)	→ Change "edge" to "vertex"	**Hamiltonian Path** Visit every vertex once; no known efficient algorithm. NP-Complete
Shortest Path Find the shortest route between two nodes (Dijkstra). P — O(E log V)	→ Change "shortest" to "longest"	**Longest Path** Find the longest simple path; becomes NP-Complete. NP-Complete
Bipartite Matching Pair elements from two sets; solved by augmenting paths. P — O(E√V)	→ Change "2 sets" to "3 sets"	**3D Matching** Match triples from three sets; NP-Complete. NP-Complete

! Complexity is **fragile**. A problem in **P** can become **NP-Complete** by changing a single constant, swapping one word, or adding one dimension. Precise definitions matter.

PART 5 · PROOF METHODOLOGY

The Two-Step Recipe for NP-Completeness Proofs

01 Show L ∈ NP

a. **Define a certificate** — a short "proof" that the answer is YES. It must be **polynomial in size**.

b. **Describe a verifier** — an algorithm that checks the certificate.

c. **Prove the verifier runs in polynomial time** — state complexity explicitly.

Goal: Establish that solutions can be checked quickly.

02 Show L is NP-Hard

a. **Choose a known NP-Complete problem L'** (e.g., 3-SAT, Vertex Cover).

b. **Construct a reduction** f that transforms any instance of L' into L in polynomial time.

c. **Prove correctness both ways:** show the iff relationship between instances.

Goal: Show L is at least as hard as a known NP-Complete problem.

APPLIED EXAMPLE

Prove Vertex Cover is NP-Complete

STEP 1 — VERTEX COVER ∈ NP

Certificate: A set $C \subseteq V$ with $|C| \leq k$.

Verifier: For each edge $(u,v) \in E$, check $u \in C$ or $v \in C$.

```
for each edge (u,v) in E:
  if u ∉ C and v ∉ C:
    return REJECT
return ACCEPT
```

Time: $O(|E| \times |C|)$ — polynomial. ✓

STEP 2 — REDUCE FROM INDEPENDENT SET

Reduction: $(G, k) \to (G, n-k)$. Then S indep. of size k ↔ V\S vertex cover of size n−k. **Time:** O(1).

Both steps complete → Vertex Cover is **NP-Complete**

! COMMON MISTAKES TO AVOID

Wrong direction / One-way proof: Reduce FROM a known NP-C problem TO yours and prove both directions.

Forgetting Step 1 / Non-poly reduction: Always prove L ∈ NP and ensure the reduction runs in polynomial time.

PART 5 · THE REDUCTION CHAIN

How NP-Completeness Proofs Build on Each Other

Each arrow means "reduces to" — proving the target is at least as hard as the source.

THE KEY PRINCIPLE

To prove a new problem X is NP-Complete, you only need to reduce **one known NP-Complete problem** to X. Because NP-Completeness is **transitive** through reductions: if $A \leq_p B$ and $B \leq_p X$, then $A \leq_p X$.

THE CHAIN KEEPS GROWING

1	21	3,000+
COOK, 1971	KARP, 1972	KNOWN TODAY

PART 5 · THE OPEN PROBLEM

The Million-Dollar Question: Does P Equal NP?

OPEN PROBLEM — UNSOLVED SINCE 1971

Does every problem whose solution can be verified in polynomial time also have a polynomial-time algorithm to find that solution?

P = NP ?

SCENARIO A

P = NP

- → **Cryptography collapses.** Public-key systems become breakable.
- → **Optimization is solved.** Scheduling and routing become trivial.
- → **AI leaps forward.** Learning and theorem-proving improve dramatically.

"Revolutionary and terrifying"

VS

SCENARIO B — MOST EXPERTS BELIEVE THIS

P ≠ NP

- → **Inherent hardness exists.** Some problems are harder to solve than to verify.
- → **Cryptography is safe.** Security rests on solid foundations.
- → **Heuristics are necessary.** Approximations remain essential.

"Confirms the world as we know it"

$1M

PRIZE MONEY

P vs NP is one of the **seven Millennium Prize Problems**. A correct proof (either direction) earns a $1,000,000 prize.

Posed: **1971**

Unsolved: **55 years**

Expert poll: **83%**

PART 5 · IF P = NP

The Consequences Would Be Revolutionary (and Terrifying)

If someone proved P = NP **constructively** — providing an actual algorithm — here's what changes overnight.

Cryptography Collapses

The foundation of internet security shatters

TODAY **RSA encryption** relies on factoring large numbers being hard. Your bank, email, and passwords are safe because no one can factor 2048-bit numbers quickly.

IF P=NP **Factoring becomes polynomial.** Every encrypted message ever sent could be decrypted. Online banking, military comms, blockchain — all broken.

RSA-2048 cracking: **~10^{600} years** now → **seconds**

Optimization Becomes Trivial

Every scheduling and routing problem is solved

TODAY **Airlines, logistics, chip design** all use heuristics and approximations because optimal solutions are NP-Hard. Results are "good enough" but never provably best.

IF P=NP **Perfect solutions in polynomial time.** Optimal airline schedules, shortest delivery routes, minimal chip layouts — all computed exactly and efficiently.

Global savings: estimated **$100B+/year** in logistics alone

AI Takes a Quantum Leap

Learning and reasoning become polynomial

TODAY **Training neural networks** is NP-Hard in general. We use gradient descent (a heuristic) and hope for good local minima. No guarantee of finding the best model.

IF P=NP **Optimal models found efficiently.** Perfect classifiers, planners, and decision-makers. AI could solve any well-defined problem with a verifiable solution.

Model training: **days of GPU** → **minutes on laptop**

Mathematics Is Automated

Finding proofs becomes as easy as checking them

TODAY **Discovering proofs** requires human creativity, intuition, and often decades of work. Verifying a proof is mechanical — just check each step.

IF P=NP **Computers find proofs automatically.** The Riemann Hypothesis, Goldbach's Conjecture — a computer could resolve them all. Human mathematicians become obsolete.

Proof search: **centuries of effort** → **polynomial time**

> *If P = NP, then the world would be a profoundly different place than we usually assume it to be. There would be* **no special value in creative leaps**, *no fundamental gap between solving a problem and recognizing the solution once it's found.*
>
> — Scott Aaronson, MIT / UT Austin

PART 6 · PRACTICAL STRATEGIES

What Do We Do When Problems Are NP-Hard?

NP-Hard doesn't mean "give up." It means **exact optimal solutions in all cases** are unlikely — but we have a powerful toolkit of practical strategies.

01 Approximation Algorithms

Find a solution **guaranteed** to be within a known factor of the optimal answer.

EXAMPLE

Vertex Cover: Greedy gives a solution ≤ **2×** optimal. Always. Provably.

⚖ Proven quality bound, but not exact.

02 Heuristics & Metaheuristics

Use clever rules of thumb that **often work well** in practice, without formal guarantees.

EXAMPLE

TSP: Simulated annealing finds near-optimal tours. Genetic algorithms for scheduling.

⚖ Great in practice, no worst-case guarantee.

03 Parameterized Complexity (FPT)

If a natural parameter **k** is small, the exponential blowup hits only k, not the full input.

EXAMPLE

Vertex Cover: $O(2^k \cdot n)$. If k=20, that's ~$10^6 \cdot n$ — fast!

⚖ Only helps when parameter is small.

04 Exploit Special Structure

Restrict the input to exploit mathematical structure. Many NP-Hard problems become **polynomial on structured inputs** like trees, planar graphs, or bounded-width instances.

EXAMPLES

2-SAT is in P (only 3-SAT is NP-C). **Graph coloring** on trees is O(n). **TSP** on Euclidean plane has a PTAS.

⚖ Only applicable when your input has the right structure.

05 Randomized Algorithms

Use randomness to find good solutions quickly. Probabilistic methods can bypass worst-case barriers and provide **expected-time guarantees** that deterministic methods cannot.

EXAMPLES

MAX-CUT: Random assignment achieves ≥ **50%** of optimal. **SAT solvers**: Random restarts dramatically improve performance.

⚖ Probabilistic guarantees — may need multiple runs.

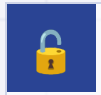
REALITY CHECK

In industry, NP-Hard problems are solved **every day** — Google Maps routes packages, airlines schedule crews, chip designers place circuits. The key insight: **worst-case hardness doesn't mean every instance is hard.** Real-world instances often have structure that algorithms can exploit.

PART 6 · REAL-WORLD IMPACT

Complexity Theory in Your Daily Life

You interact with NP-Hard problems **every single day** — you just don't realize it. Here's a typical student's day, decoded.



7:30 AM

Unlock Your Phone

Every HTTPS connection, every password hash, every encrypted message relies on **integer factoring being hard**.

Factoring / RSA ASSUMED HARD

Strategy: **Security by hardness assumption**

8:15 AM

Google Maps Route

Finding the optimal route through traffic is a **variant of TSP**. Google uses heuristics on road-network graphs with special structure.

Vehicle Routing (TSP) NP-HARD

Strategy: **Heuristics + graph structure**

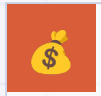
9:00 AM

Course Timetable

Assigning courses to rooms and time slots with no conflicts is **graph coloring**. Universities use constraint solvers.

Graph Coloring NP-COMPLETE

Strategy: **Constraint programming**



12:30 PM

Split the Bill Fairly

Dividing shared expenses so everyone pays their fair share is a **partition problem**. Small groups make it tractable.

Subset Sum / Partition NP-COMPLETE

Strategy: **Small input → exact solution**

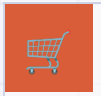
7:00 PM

Netflix Recommendations

Recommending the best content from millions of options is an **optimization problem**. Netflix uses matrix factorization and ML approximations.

Combinatorial Optimization NP-HARD

Strategy: **Approximation + ML heuristics**



9:30 PM

Online Shopping Deals

Maximizing value within a budget from a sale catalog is the **Knapsack problem**. Small catalogs use dynamic programming.

0/1 Knapsack NP-COMPLETE

Strategy: **Pseudo-polynomial DP**

KEY TAKEAWAY

Complexity theory isn't abstract — it's the reason your **passwords are secure**, your **GPS works**, and your **university schedule exists**. Every time an algorithm makes a "good enough" decision quickly, it's because someone understood **why finding the perfect answer is hard**.

PART 7 · COMMON PITFALLS

What Students Often Get Wrong About Complexity Classes

These six misconceptions appear on exams **every semester**. Make sure you don't fall for them.

01 "NP" Doesn't Mean What You Think

✗ ~~"NP stands for Not Polynomial — these problems can't be solved in polynomial time."~~

✓ NP stands for **Nondeterministic Polynomial**. It means solutions can be **verified** in polynomial time. Many NP problems **are** in P (e.g., sorting, shortest path).

02 NP-Hard ≠ Impossible to Solve

✗ ~~"NP-Hard problems can never be solved — they're unsolvable."~~

✓ NP-Hard problems **can be solved**, just not efficiently in the worst case. We solve them **every day** with heuristics, approximations, and SAT solvers.

03 Being in NP Doesn't Mean Hard

✗ ~~"If a problem is in NP, it must be hard to solve."~~

✓ **P ⊆ NP** — every easy problem is also in NP. Sorting, searching, shortest path are all in NP. Only the **NP-Complete** problems at the boundary are believed to be hard.

04 NP-Complete Isn't the Hardest

✗ ~~"NP-Complete is the hardest complexity class there is."~~

✓ NP-Hard problems **outside NP** are even harder — like the **Halting Problem** (undecidable). NP-Complete problems are the hardest **within NP**, not overall.

05 Exponential Time ≠ NP

✗ ~~"NP means the problem requires exponential time to solve."~~

✓ NP is about **verification time**, not solving time. Some NP problems are in P. Some problems requiring exponential time (like EXPTIME-complete) are **outside** NP entirely.

06 The Gap Between P and NP-C

✗ ~~"Every NP problem is either in P or NP-Complete — nothing in between."~~

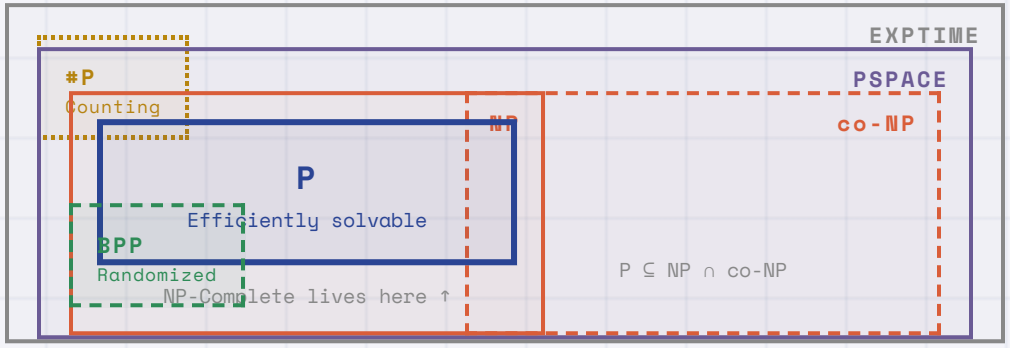
✓ **Ladner's Theorem** (1975): If P ≠ NP, there **must** exist problems in NP that are neither in P nor NP-Complete. **Graph Isomorphism** is a leading candidate.

PART 7 · THE BIGGER PICTURE

Beyond P and NP: Other Complexity Classes

P and NP are just two rooms in a vast mansion. Here's a map of the neighborhood.

CONTAINMENT HIERARCHY (ASSUMED P ≠ NP)



* Exact containments between NP, co-NP, PSPACE are open questions

co-NP Problems where **NO** answers have short, verifiable proofs.

EXAMPLE
UNSAT — prove a formula has no satisfying assignment

KEY FACT
If **NP = co-NP**, then P = NP (likely false)

PSPACE Solvable with **polynomial memory**, but possibly exponential time.

EXAMPLE
QBF — quantified boolean formulas; generalized chess on n×n board

KEY FACT
Contains both NP and co-NP: **NP ⊆ PSPACE**

EXPTIME Solvable in **exponential time** — the "brute force" ceiling.

EXAMPLE
Generalized **Go** on n×n board; full game trees

KEY FACT
P ≠ EXPTIME — one separation we can actually prove!

BPP Solvable in poly-time with **randomness** and bounded error ≤ 1/3.

EXAMPLE
Primality testing (Miller-Rabin); polynomial identity testing

KEY FACT
Widely believed **BPP = P** (randomness doesn't help much)

#P **Counting** problems — how many solutions exist, not just "does one?"

EXAMPLE
#SAT — count satisfying assignments; permanent of a matrix

KEY FACT
Harder than NP — even for problems in P, counting can be **#P-hard**

WHY IT MATTERS These classes reveal that **computational difficulty has many dimensions** — time, space, randomness, counting. Understanding where your problem sits in this hierarchy tells you **which tools and strategies** are available and which are provably insufficient.

SUMMARY · KEY TAKEAWAYS

What You Should Remember from Today

01 **P = Efficiently Solvable**

Problems solvable in **polynomial time**. Sorting, shortest path, matching — all in P.

02 **NP = Efficiently Verifiable**

Solutions can be **checked** in polynomial time. Every problem in P is also in NP.

03 **NP-Complete = Hardest in NP**

Solve **one** efficiently → solve **all of NP** efficiently. 3-SAT, TSP, Knapsack are all equivalent.

04 **NP-Hard = At Least as Hard**

As hard as NP-Complete, but **might not be in NP**. Includes optimization and undecidable problems.

05 **Reductions Are the Key Tool**

Prove NP-Completeness by reducing **FROM** a known NP-C problem **TO** yours. Direction matters.

06 **P vs NP Is Still Open**

Most experts believe **P ≠ NP**, but no one has proven it. A **$1 million** prize awaits.

07 **Small Changes Flip Complexity**

2-SAT → 3-SAT, shortest → longest path, 2-coloring → 3-coloring. **One tweak** changes everything.

08 **NP-Hard ≠ Impossible**

Approximation, heuristics, FPT, and special cases solve NP-Hard problems **every day** in practice.

THE ONE SENTENCE TO REMEMBER

The central insight of complexity theory is that *verifying* a solution can be fundamentally easier than finding one — and whether this gap is real remains the greatest open question in computer science.

CLOSING · FURTHER READING

Continue Your Exploration

This lecture is just the beginning. Complexity theory is a **living, active field** with deep connections to cryptography, AI, and mathematics. Here's how to go deeper.

📚 Books & Textbooks

Introduction to the Theory of Computation
The standard undergraduate textbook — clear and rigorous.
Essential

Computers and Intractability
The classic NP-completeness reference with many reductions.
Essential

Computational Complexity: A Modern Approach
Graduate-level deep dive; free draft online.
Advanced

Quantum Computing Since Democritus
An entertaining tour from logic to quantum computing.
Fun Read

🌐 Online Resources

MIT OCW 6.045J
Full MIT course: Automata, Computability, and Complexity.
Essential

Complexity Zoo
Encyclopedia of complexity classes with definitions and links.
Advanced

P vs NP — Hackerdashery
A concise animated explainer of P vs NP.
Fun Watch

Shtetl-Optimized Blog
Scott Aaronson's commentary on complexity and quantum topics.
Fun Read

🎯 Practice & Next Steps

Complete the Online Exercise
Take the interactive quiz linked from this lecture.
Essential

Practice NP-Completeness Proofs
Try reductions on paper (3-SAT → Independent Set, etc.).
Essential

Algorithm Design Course
Follow up with approximation and randomized algorithms.
Next Course

Competitive Programming
Practice on Codeforces / LeetCode to build intuition.
Fun Practice

Thank You for Your Attention

The boundary between **easy** and **hard** remains a deep mystery — see the **reference document** for worked examples.

OPEN FLOOR

?

Questions